Injecting Dependencies into Controllers

Laravel's facades present a simple interface to the most useful classes in Laravel's codebase. You can get information about the current request and user input, the session, caches, and much more.

But if you prefer to inject your dependencies, or if you want to use a service that doesn't have a facade, you'll need to find some way to bring instances of these classes into your controller.

This is our first exposure to Laravel's service container. For now, if this is unfamiliar, you can think about it as a little bit of Laravel magic; or, if you want to know more about how it's actually functioning, you can skip ahead to Chapter 11.

All controller methods (including the constructors) are resolved out of Laravel's container, which means anything you typehint that the container knows how to resolve will be automatically injected.

As a nice example, what if you'd prefer having an instance of the `Request` object instead of using the facade? Just typehint `Illuminate\Http\Request` in your method parameters, like in Example 3-25.

Example 3-25. Controller method injection via typehinting

```
// TasksController.php
...
public function store(\Illuminate\Http\Request $request)
{
    $task = new Task;
    $task->title = $request->input('title');
    $task->description = $request->input('description');
    $task->save();

    return redirect('tasks');
}
```

So, you've defined a parameter that must be passed into the `store()` method. And since you typehinted it, and since Laravel knows how to resolve that class name, you're going to have the `Request` object ready for you to use in your method with no work on your part. No explicit binding, no anything else — it's just there as the `$request` variable.

By the way, this is actually how I and many other Laravel developers prefer to get the user input: inject an instance of the `Request` and read the user input from there, instead of relying on the `Input` facade.

Resource Controllers

Sometimes naming the methods in your controllers can be the hardest part of writing a controller. Thankfully, Laravel has some conventions for all of the routes of a traditional REST/CRUD controller (called a "resource controller" in Laravel); additionally, it comes with a generator out of the box and a convenience route definition that allows you to bind an entire resource controller at once.

To see the methods that Laravel expects for a resource controller, let's generate a new controller from the command line:

```
php artisan make:controller MySampleResourceController --resource
```

Now open *app/Http/Controllers/MySampleResourceController.php*. You'll see it comes prefilled with quite a few methods. Let's walk over what each represents. We'll use a `Task` as an example.

The methods of Laravel's resource controllers

For each, you can see the HTTP verb, the URL, the controller method name, and the "name." Table 3-1 shows the HTTP verb, the URL, the controller method name, and the "name" for each of these default methods.

Table 3-1. The methods of Laravel's resource controllers

Verb	URL	Controller method	Name	Description
`GET`	`tasks`	`index()`	`tasks.index`	Show all tasks
`GET`	`tasks/create`	`create()`	`tasks.create`	Show the create task form
`POST`	`tasks`	`store()`	`tasks.store`	Accept form submission from the create task form
`GET`	`tasks/{task}`	`show()`	`tasks.show`	Show one task
`GET`	`tasks/{task}/edit`	`edit()`	`tasks.edit`	Edit one task
`PUT/PATCH`	`tasks/{task}`	`update()`	`tasks.update`	Accept form submission from the edit task form
`DELETE`	`tasks/{task}`	`destroy()`	`tasks.destroy`	Delete one task

Binding a resource controller

So, we've seen that these are the conventional route names to use in Laravel, and also that it's easy to generate a resource controller with methods for each of these default routes. Thankfully, you don't have to generate routes for each of these controller methods by hand, if you don't want to. Instead, there's a trick for that, and it's called "resource controller binding." Take a look at Example 3-26.

Example 3-26. Resource controller binding

```
// routes/web.php
Route::resource('tasks', 'TasksController');
```

This will automatically bind all of the routes for this resource to the appropriate method names on the specified controller. It'll also name these routes appropriately; for example, the `index()` method on the `tasks` resource controller will be named `tasks.index`.

ARTISAN ROUTE:LIST

If you ever find yourself in a situation where you're wondering what routes your current application has available, there's a tool for that: from the command line, run `php artisan route:list` and you'll get a listing of all of the available routes (see Figure 3-2).

```
mattstauffer at Cassim in ~/Sites/book-up-and-running
o php artisan route:list
+--------+----------+-----------------+--------------+-----------------------------------------+--------------+
| Domain | Method   | URI             | Name         | Action                                  | Middleware   |
+--------+----------+-----------------+--------------+-----------------------------------------+--------------+
|        | GET|HEAD | /               |              | Closure                                 | web          |
|        | GET|HEAD | api/user        |              | Closure                                 | api,auth:api |
|        | GET|HEAD | dogs            | dogs.index   | App\Http\Controllers\DogsController@index   | web          |
|        | POST     | dogs            | dogs.store   | App\Http\Controllers\DogsController@store   | web          |
|        | GET|HEAD | dogs/create     | dogs.create  | App\Http\Controllers\DogsController@create  | web          |
|        | GET|HEAD | dogs/{dog}      | dogs.show    | App\Http\Controllers\DogsController@show    | web          |
|        | PUT|PATCH | dogs/{dog}      | dogs.update  | App\Http\Controllers\DogsController@update  | web          |
|        | DELETE   | dogs/{dog}      | dogs.destroy | App\Http\Controllers\DogsController@destroy | web          |
|        | GET|HEAD | dogs/{dog}/edit | dogs.edit    | App\Http\Controllers\DogsController@edit    | web          |
+--------+----------+-----------------+--------------+-----------------------------------------+--------------+
```

Figure 3-2. php artisan route:list example